

فتح اتصالاً برمجياً مع السيرفر ويمرر المعايير الأربعة الأساسية لربط قاعدة البيانات باستخدام دالة
(mysql_connect())

خطوات كتابة كود فتح الاتصال بقاعدة البيانات:(mysql_connect())

1. حجز متغير الاتصال: تبدأ الجملة البرمجية بكتابة رمز المتغير في لغة PHP والمخصص لحمل بيانات قنوات الربط مثل \$conn .

```
$conn =
```

2. استدعاء دالة الاتصال المباشر: ترك مسافة فارغة واحدة، ثم كتابة اسم الدالة القياسية بأحرف صغيرة mysql_connect متبوعة بفتح قوسين هلاليين () .

```
$conn = mysql_connect();
```

3. تمرير معايير الخادم والمستخدم: كتابة المعيار الأول وهو عنوان السيرفر المحرك بين علامات تنصيص متبوعاً بفاصلة، ثم المعيار الثاني وهو اسم مستخدم الصلاحيات مثل 'localhost', 'root'.
4. تمرير معايير الحماية واسم المستودع: كتابة المعيار الثالث وهو كلمة المرور متبوعاً بفاصلة، ثم المعيار الرابع والأخير وهو الاسم الصريح لقاعدة البيانات المستهدفة مثل 'school_db',

```
$conn = mysql_connect('localhost', 'root', '', 'school_db');
```

5. إغلاق جملة الربط البرمجية: وضع القوس الهلالي المغلق لإنهاء نطاق الدالة، ثم إنهاء السطر البرمجي بوضع الفاصلة المنقوطة ؛ لتأكيد جاهزية الأمر للتنفيذ.

('عنوان السيرفر', 'اسم المستخدم', 'كلمة المرور', 'اسم قاعدة البيانات') = **mysql_connect** متغير الاتصال \$

```
$conn = mysql_connect('localhost', 'root', '', 'school_db') ;
```

فحص حالة ونجاح الاتصال بالسيرفر ويعالج أخطاء الربط المحتملة باستخدام دالة الفحص (mysqli_connect_error())

خطوات كتابة كود فحص حالة الاتصال ومعالجة الأخطاء :

1. صياغة الجملة الشرطية الاختبارية: تبدأ الجملة البرمجية بكتابة أداة الشرط if بأحرف صغيرة متبوعة بفتح قوسين هلاليين وفحص نفي متغير الاتصال باستخدام علامة التعجب (مثل if (!\$conn) .

```
if (!$conn) {
```

2. استدعاء أداة الإيقاف وطباعة الخطأ: فتح قوس مجعد ثم الانتقال لسطر جديد وكتابة دالة الإيقاف الفوري die بأحرف صغيرة لقطع اتصال الصفحة عند الفشل.
3. تضمين دالة الفحص النصية: فتح قوسين هلاليين لدالة الإيقاف، وتمرير رسالة تنبيهية متبوعة بعلامة الربط متموزة باستدعاء دالة الفحص المباشر mysqli_connect_error() لجلب نص الخطأ من السيرفر.

```
if (!$conn) {  
    die(mysqli_connect_error());  
}
```

4. إغلاق جملة المعالجة وإيقاف السكربت: إنهاء أمر دالة الإيقاف بوضع الفاصلة المنقوطة ثم الانتقال لسطر جديد وإغلاق القوس المجعد للشرط {}.
5. إنهاء المعاملة الشرطية بالكامل: التأكد من سلامة بناء الأقواس الهلالية والمجعدة لضمان تشغيل السكربت بدون أخطاء بناء برمجي للطلاب في المعمل.

```
if (متغير الاتصال !$conn)  
{  
    die(mysqli_connect_error());  
}
```

```
if (!$conn)  
{  
    die(" فشل الاتصال بقاعدة البيانات ");  
}
```

توظيف الجمل المعدة مسبقاً (Prepared Statements) لتأمين البيانات وحظر هجمات الاختراق وإعادة استخدام خطط التنفيذ داخل السيرفر

خطوات كتابة وتوظيف الجمل المعدة مسبقاً: (Prepared Statements)

1. تجهيز قالب الاستعلام الهيكلي: تبدأ الجملة البرمجية بصياغة أمر SQL الأساسي وحفظه في متغير مع استبدال المدخلات بعلامات استفهام (مثل \$query = "INSERT INTO students (name) VALUES (?);").

```
$query = "INSERT INTO students (name) VALUES (?);"
```

2. تمرير الهيكل والتحضير داخل السيرفر: كتابة دالة التحضير المباشر `mysqli_prepare()` بتمرير متغير الاتصال ومتغير الاستعلام لتجهيز الخطة الأمنية وحفظها في متغير المعاملة (مثل `$stmt = mysqli_prepare($conn, $query);`).

```
$query = "INSERT INTO students (name) VALUES (?);"  
$stmt = mysqli_prepare($conn, $query);
```

3. ربط وحقن بيانات المستخدم بأمان: الانتقال لسطر جديد واستدعاء دالة الربط `mysqli_stmt_bind_param()` بتمرير متغير المعاملة متبوعاً بنوع البيانات ثم المتغير الفعلي للقيمة (مثل `mysqli_stmt_bind_param($stmt, "s", $name);`).

```
$query = "INSERT INTO students (name) VALUES (?);"  
$stmt = mysqli_prepare($conn, $query);  
mysqli_stmt_bind_param($stmt, "s", $name);
```

4. إعطاء أمر التنفيذ النهائي: الانتقال لسطر جديد واستدعاء دالة التشغيل الفعلي
mysqli_stmt_execute() متبوعة بتمرير متغير المعاملة المجهز الحاكم (مثل
. mysqli_stmt_execute(\$stmt))

```
$query = "INSERT INTO students (name) VALUES (?);"  
$stmt = mysqli_prepare($conn, $query);  
mysqli_stmt_bind_param($stmt, "s", $name);  
mysqli_stmt_execute($stmt);
```

5. إغلاق المعاملة وتأمين الذاكرة: إنهاء السطر البرمجي بوضع الفاصلة المنقوطة ؛ للإشارة إلى اكتمال
مسار التأمين وجاهزية الكود للعمل.

```
$query = "INSERT INTO students (name) VALUES (?);"  
$stmt = mysqli_prepare($conn, $query);  
mysqli_stmt_bind_param($stmt, "s", $name);  
mysqli_stmt_execute($stmt);  
mysqli_stmt_close($stmt);
```

\$query = "INSERT INTO اسم_الجدول (اسم_العمود) VALUES (?);
متغير_الاستعلام

متغير_الاتصال, \$متغير_الاستعلام) = mysqli_prepare(متغير_المعاملة

متغير_القيمة = "القيمة" ;

متغير_القيمة, "s", متغير_المعاملة) = mysqli_stmt_bind_param(متغير_المعاملة

متغير_المعاملة) = mysqli_stmt_execute(متغير_المعاملة

```
$query = "INSERT INTO students (student_name) VALUES (?);"
```

```
$stmt = mysqli_prepare($conn, $query);
```

```
$student_name = "نرمين ونيس";
```

```
mysqli_stmt_bind_param($stmt, "s", $student_name);
```

```
mysqli_stmt_execute($stmt);
```

أوامر وجمل SQL المختلفة برمجياً من داخل سكربت PHP باستخدام دالة التنفيذ (`mysqli_query()`)

خطوات كتابة كود تنفيذ الاستعلامات برمجياً: (`mysqli_query()`)

1. صياغة وحفظ النص البرمجي للاستعلام: تبدأ الجملة البرمجية بكتابة نص أمر SQL المراد تشغيله بالكامل وحفظه داخل متغير نصي مخصص مثل `$query = "SELECT * FROM students"`
2. استدعاء دالة التنفيذ وتمرير المعايير: ترك مسافة فارغة واحدة، ثم كتابة اسم متغير النتيجة متبوعاً بمعامل المساواة واستدعاء الدالة بأحرف صغيرة (مثل `$result = mysqli_query()`).
3. تضمين رابط الاتصال الحاكم: فتح قوسين هلاليين () وتمرير المتغير الخاص بقناة الاتصال المفتوحة كمعيار أول وإلحاقه بفاصلة عادية مثل `$conn`,
4. تمرير متغير الأمر المستهدف: كتابة اسم المتغير النصي الذي يحمل أمر SQL كمعيار ثانٍ لإتمام عملية التمرير مثل `$query`

```
$query = "SELECT * FROM students";  
$result = mysqli_query($conn, $query);
```

5. إغلاق جملة التنفيذ وإطلاق الأمر: غلق القوس الهلالي وإنهاء السطر البرمجي بوضع الفاصلة المنقوطة ؛ للإشارة إلى جاهزية الطلب للإرسال الفوري للسيرفر.

```
؛ "اسم الجدول * FROM" = متغير الاستعلام $  
؛ (متغير الاتصال, $متغير الاستعلام) mysqli_query = متغير النتيجة $
```

```
$query = "SELECT * FROM students";  
$result = mysqli_query($conn, $query);
```

غلق اتصال قاعدة البيانات فور انتهاء العمليات لضمان كفاءة السيرفر وتوفير الموارد باستخدام دالة `(mysqli_close())`

خطوات كتابة كود إغلاق اتصال قاعدة البيانات: `(mysqli_close())`

1. الوصول لنهاية العمليات التنفيذية للسكريبت: تبدأ الخطوة البرمجية بالنزول إلى السطر الأخير في صفحة الأكواد بعد الانتهاء التام من التعامل مع البيانات.
2. استدعاء دالة قطع الربط القياسية: كتابة اسم الدالة الحاكمة بأحرف صغيرة `mysqli_close` لبدء مسار تحرير الموارد المعلقة.
3. تضمين متغير الاتصال المستهدف: فتح قوسين هلاليين () مباشرة بعد الدالة، وتمرير اسم المتغير الفعلي الذي يحمل معرف الاتصال المفتوح مثل `$conn`

```
mysqli_close($conn);
```

4. إغلاق القوس البرمجي للدالة: غلق القوس الهلالي جهة اليمين لحصر نطاق التمرير ومطابقة البناء النحوي للغة PHP.
5. إنهاء السطر وإطلاق أمر التحرير: وضع الفاصلة المنقوطة ; في نهاية السكريبت للإشارة إلى نهاية مسار الأمر وجاهزيته للتنفيذ.

؛ (متغير الاتصال \$) `mysqli_close`

```
mysqli_close($conn);
```

استدعاء الإجراءات المخزنة (Stored Procedures) لتنفيذ عمليات معالجة البيانات المتكررة مباشرة داخل السيرفر

خطوات كتابة واستدعاء الإجراء المخزن: (Stored Procedure)

1. بناء وتأسيس الإجراء داخل السيرفر: تبدأ العملية داخل بيئة MySQL بكتابة الأمر CREATE PROCEDURE متبوعاً باسم الإجراء وقوسين، ثم كتلة العمل بين BEGIN و END وحفظها داخل القواعد.

2. صياغة أمر الاستدعاء النصي في: PHP الانتقال إلى سكربت PHP وكتابة أمر النداء القياسي باستخدام الكلمة المحجوزة CALL متبوعة باسم الإجراء المحفوظ، وتخزينه في متغير مستقل مثل \$query = "CALL GetTopStudents()".

```
$query = "CALL GetTopStudents()";
```

3. تمرير الأمر لدالة التنفيذ الإجرائية: تترك مسافة فارغة واحدة، ثم كتابة متغير النتيجة متبوعاً بدالة التشغيل الفوري mysqli_query() وتمرير متغير الاتصال ومتغير الاستعلام بداخلها.

```
$query = "CALL GetTopStudents()";  
$result = mysqli_query($conn, $query);
```

4. تخزين مخرجات الإجراء وعرضها: التأكد من استقبال مخرجات الدالة بالكامل داخل متغير النتيجة الحاضن مثل \$result استعداداً لمعالجة السجلات.

```
$query = "CALL GetTopStudents()";  
$result = mysqli_query($conn, $query);  
if ($result) {  
    // معالجة وطباعة بيانات الطلاب هنا  
}
```

5. إغلاق جملة المعاملة البرمجية: إنهاء السطر البرمجي بالكامل بوضع الفاصلة المنقوطة ؛ للإشارة إلى اكتمال البناء الهيكلي.

\$متغير الإنشاء =

"CREATE PROCEDURE اسم الإجراء

BEGIN

استعلام الفلترة والتصفية لقاعدة البيانات

END";

@mysqli_query(\$متغير الإتصال,\$متغير الإنشاء);

\$متغير الاستعلام = "CALL اسم الإجراء المخزن";

\$query_create =

"

CREATE PROCEDURE **GetTopStudents()**

BEGIN

SELECT student_name, grade_score FROM students WHERE
grade_score >= 90;

END";

@**mysqli_query**(\$conn, **\$query_create**);

\$query = "CALL **GetTopStudents()**";

\$result = **mysqli_query**(\$conn, \$query);

استدعاء الدوال البرمجية (Stored Functions) لإجراء العمليات الحسابية المعقدة وإرجاع قيم محددة للنظام

خطوات كتابة واستدعاء الدالة البرمجية المخزنة: (Stored Function)

1. بناء وتأسيس الدالة داخل السيرفر: تبدأ العملية داخل بيئة MySQL بكتابة الأمر CREATE FUNCTION متبوعاً باسم الدالة ومعاملاتها، مع تحديد نوع القيمة العائدة باستخدام العبارة RETURNS وصياغة الكود الحسابي بين BEGIN و END .
2. صياغة أمر الاستدعاء داخل استعلام PHP: الانتقال إلى سكربت PHP وكتابة جملة SELECT القياسية متبوعة مباشرة باستدعاء اسم الدالة وتمرير القيم المراد حسابها بين قوسين، وحفظ السكربت داخل متغير مثل \$query = "SELECT CalculateTax(100)";

```
$query = "SELECT CalculateTax(100) AS TaxValue";
```

3. تمرير الاستعلام لدالة التنفيذ: ترك مسافة فارغة واحدة، ثم كتابة متغير النتيجة متبوعاً بدالة التشغيل الفوري mysqli_query() وتمرير متغير الاتصال ومتغير الاستعلام بداخلها لإرسال الطلب إلى السيرفر .

```
$query = "SELECT CalculateTax(100) AS TaxValue";  
$result = mysqli_query($conn, $query);
```

4. تخزين القيمة الحسابية العائدة: فتح مساحة متغير داخل الذاكرة مثل \$result لحفظ الناتج الحسابي المفرد المرتجع من السيرفر نتيجة لتنفيذ الدالة.

```
$query = "SELECT CalculateTax(100) AS TaxValue";  
$result = mysqli_query($conn, $query);  
if ($result) {  
    $row = mysqli_fetch_assoc($result);  
    echo " قيمة الضريبة هي: " . $row['TaxValue'];  
}
```

5. إغلاق جملة المعاملة الحسابية: إنهاء السطر البرمجي بوضع الفاصلة المنقوطة ؛ للإشارة إلى اكتمال البناء الهيكلي وجاهزيته للتشغيل.

\$متغير الإنشاء =

" CREATE FUNCTION (المتعامل نوع_البيانات) اسم الدالة

RETURNS نوع_البيانات

DETERMINISTIC

BEGIN

RETURN المعادلة الحسابية أو القيمة المرجعة

END";

@mysqli_query(\$متغير الإنشاء, متغير الاتصال);

\$متغير الاستعلام = "SELECT (المتعامل)المخزنة اسم الدالة";

\$متغير الاتصال, \$متغير الاستعلام)mysqli_query(= متغير النتيجة

\$query_create =

" CREATE FUNCTION CalculateTax(price INT)

RETURNS INT

DETERMINISTIC

BEGIN

RETURN price * 0.15;

END";

@mysqli_query(\$conn, \$query_create);

\$query = "SELECT

CalculateTax(100) AS total_tax";

\$result = mysqli_query(\$conn, \$query);

إدارة المعاملات البرمجية المتتالية وضمان سلامة تنفيذها بالكامل أو تراجعها التام عند حدوث خطأ عبر مفهوم (Transaction Concept)

خطوات كتابة وإدارة المعاملات البرمجية المتتالية: (Transaction)

1. إيقاف ميزة التثبيت التلقائي للسيرفر :تبدأ الجملة البرمجية باستدعاء الدالة القياسية `mysqli_autocommit()` وتميرير رابط الاتصال مع القيمة `false` لتأجيل حفظ الأوامر مؤقتاً.

```
mysqli_autocommit($conn, false);
```

2. صياغة الأوامر المترابطة وتنفيذها :كتابة الأوامر المتتالية وتميريرها عبر دالة التنفيذ المعتادة `mysqli_query()` وحفظ نتائجها في متغيرات مستقلة مثل `$res1` و `$res2`

```
mysqli_autocommit($conn, false);  
$res1 = mysqli_query($conn, $query1);  
$res2 = mysqli_query($conn, $query2);
```

3. فحص ومراقبة سلامة الأكواد :بناء جملة شرطية اختبارية باستخدام أداة الشرط `if` تتضمن تحقق نجاح جميع الأوامر السابقة معاً دون حدوث أي خطأ فني.

```
mysqli_autocommit($conn, false);  
$res1 = mysqli_query($conn, $query1);  
$res2 = mysqli_query($conn, $query2);  
if ($res1 && $res2) {
```

4. تأكيد التثبيت النهائي عند النجاح المطلق :استدعاء دالة الحفظ الصريح `mysqli_commit()` بأحرف صغيرة لتأصيل البيانات داخل الجداول بشكل دائم في حال عبور الشرط بأمان.

```
mysqli_autocommit($conn, false);  
$res1 = mysqli_query($conn, $query1);  
$res2 = mysqli_query($conn, $query2);  
if ($res1 && $res2) {  
    mysqli_commit($conn);
```

5. إطلاق أمر التراجع الشامل عند الفشل الفوري: كتابة أمر التراجع `mysqli_rollback()` داخل مسار الفشل البديل `else` لإلغاء كافة الخطوات السابقة فوراً وإعادة القواعد لحالتها الأصلية، ثم إنهاء السطور بالفاصلة المنقوطة; .

```
mysqli_autocommit($conn, false);
$res1 = mysqli_query($conn, $query1);
$res2 = mysqli_query($conn, $query2);
if ($res1 && $res2) {
    mysqli_commit($conn);
} else {
    mysqli_rollback($conn);
}
```

```
mysqli_autocommit($متغير الاتصال, false);
$متغير الاتصال, $متغير الاستعلام الأول) = mysqli_query($النتيجة الأول);
$متغير الاتصال, $متغير الاستعلام الثاني) = mysqli_query($النتيجة الثاني);
if ($متغير النتيجة الثاني && $متغير النتيجة الأول)
{
    mysqli_commit($متغير الاتصال);
}
else {
    mysqli_rollback($متغير الاتصال);
}
```

```
mysqli_autocommit($conn, false);
$query1 = "UPDATE
accounts SET balance = balance - 100 WHERE user_id = 1";
$res1 = mysqli_query($conn, $query1);
$query2 = "UPDATE
accounts SET balance = balance + 100 WHERE user_id = 2";
$res2 = mysqli_query($conn, $query2);
if ($res1 && $res2)
{
    mysqli_commit($conn);
}
else {
    mysqli_rollback($conn);
}
```

معالجة النصوص المدخلة لتأمينها من هجمات حقن القواعد (SQL Injection) باستخدام دالة (mysqli_real_escape_string())

تُعد هذه المهارة الأداة البرمجية الوقائية الأساسية لحماية المواقع، حيث يتمثل دورها الوظيفي في تنظيف وتطهير النصوص والبيانات القادمة من مستخدم الموقع قبل إرسالها إلى قاعدة البيانات، عن طريق إبطال مفعول أي رموز أو علامات تنصيب خاصة قد تُستغل لتغيير مسار الأوامر. وتكمن الأهمية التعليمية والعملية لتطبيق دالة (mysqli_real_escape_string()) في تدريب الطالب على تأمين تطبيقات الويب ضد هجمات اختراق وتدمير القواعد المعروفة باسم (SQL Injection) ؛ وتعتمد آلية عملها على استقبال معيارين: رابط الاتصال الفعال والنص المراد تنظيفه، ثم تقوم بإضافة شرطة مائلة خلفية (\) تلقائياً قبل أي علامة تنصيب مدخلة ليعاملها السيرفر كنص مجرد لا كأمر برمجي.

خطوات كتابة كود معالجة وتأمين النصوص المدخلة: (mysqli_real_escape_string())

1. **حجز متغير لاستقبال النص المؤمن**: تبدأ الجملة البرمجية بكتابة رمز المتغير الجديد في لغة PHP

والمخصص لحفظ النص بعد تطهيره مثل \$secure_name =

```
$secure_name =
```

2. **استدعاء دالة التنظيف القياسية**: ترك مسافة فارغة واحدة، ثم كتابة اسم الدالة بأحرف صغيرة

mysqli_real_escape_string متبوعة بفتح قوسين هلاليين () .

```
$secure_name = mysqli_real_escape_string($conn, $user_name);
```

3. **تضمين رابط الاتصال الحاكم كمعيار أول**: كتابة اسم المتغير الخاص بقناة الاتصال المفتوحة والفعالة

مع السيرفر يليه فاصلة عادية مثل \$conn,

4. **تمرير المتغير المراد تطهيره كمعيار ثانٍ**: كتابة اسم المتغير النصي الأصلي والقادم من نموذج

المدخلات والمطلوب تأمينه مثل \$user_name

5. **إغلاق الدالة وإنهاء السطر البرمي**: غلق القوس الهلالي جهة اليمين، ثم وضع الفاصلة المنقوطة ;

للإشارة إلى اكتمال مسار التطهير وجاهزية المتغير الآمن للاستخدام.

= متغير النص المؤمن \$

mysqli_real_escape_string(\$متغير الاتصال, \$متغير النص الأصلي);

```
$secure_name =
```

```
mysqli_real_escape_string($conn, $user_name);
```

جملته الحفظ INSERT INTO ويربطها بأسماء الحقول والقيم المستهدفة بدقة داخل السكريبت

تعد هذه المهارة الأسلوب البرمجي الدقيق لبناء عبارات التغذية التفاعلية، حيث يتمثل دورها الوظيفي في صياغة نص أمر الإدراج وتنسيقه هيكلياً داخل سكريبت الـ PHP ليربط بين الأعمدة الفعلية للجدول والقيم المؤمنة المستخلصة من واجهة الموقع. وتكمن الأهمية التعليمية والعملية لتطبيق هذا النمط في تمكين الطالب من صياغة جمل استعلام ديناميكية مرنة وقابلة للتغير، بدلاً من الجمل الثابتة، لضمان دمج البيانات القادمة من نماذج الويب (HTML Forms) بدقة وعناية؛ وتعتمد آلية عملها على حجز مساحة متغير نصي يُكتب بداخله السكريبت البرمي للـ SQL مع وضع المتغيرات الحاضرة للقيم داخل علامات تنصيص مفردة مريحة للمحرك.

خطوات صياغة جملة الحفظ والربط الدقيق داخل السكريبت:

1. حجز متغير نصي لحفظ الاستعلام: تبدأ الجملة البرمجية بكتابة المتغير المخصص لحمل نص الأمر

البرمي بالكامل متبوعاً بعلامة المساواة مثل = \$query

```
$query = "INSERT INTO students (student_name) VALUES ('$secure_name')";
```

2. فتح علامات التنصيص الحاضرة للأمر: وضع علامتي تنصيص مزدوجة "" للإعلان عن بدء كتابة

السكريبت النصي الخاص بقواعد البيانات.

3. كتابة أمر التغذية وتحديد الجدول والأعمدة: كتابة العبارة المحجوزة INSERT INTO متبوعة باسم

الجدول، ثم فتح قوسين وتوصيف الأعمدة المستهدفة مثل INSERT INTO students (student_name)

4. ربط وحقق المتغيرات المؤمنة بالتوالي: الانتقال لسطر جديد وكتابة الكلمة المفتاحية VALUES متبوعة بفتح قوسين ووضع اسم متغير القيمة المؤمنة بين علامات تنصيص مفردة (مثل VALUES ('\$secure_name'))

5. إغلاق جملة الصياغة النصية: وضع الفاصلة المنقوطة الداخلية لـ SQL ثم غلق علامة التنصيص المزدوجة وإنهاء السطر بوضع الفاصلة المنقوطة للغة;.

= متغير الاستعلام \$

;"(متغير النص المؤمن '\$') VALUES اسم الجدول (اسم العمود) INSERT INTO

\$query =

;"INSERT INTO students (student_name) VALUES ('\$secure_name');

أمر الحفظ والارسال إلى السيرفر لإتمام تخزين البيانات باستخدام دالة التنفيذ mysqli_query()

تعد هذه المهارة الأداة التنفيذية الختامية لنقل وتأسيس البيانات، حيث يتمثل دورها الوظيفي في إطلاق وإرسال أمر الحفظ الذي تم صياغته وتأمينه مسبقاً، وضخه برمجياً عبر قناة الاتصال المفتوحة ليقوم خادم MySQL بتسجيله وتحديث مستودع البيانات فوراً. وتكمن الأهمية التعليمية والعملية لتطبيق دالة mysqli_query() في هذا السياق في جعل الطالب يرى النتيجة الفعلية لتفاعل واجهات الويب مع السيرفر (مثل اكتمال عملية تسجيل مستخدم جديد بنجاح وظهور بياناته في الجدول)؛ وتعتمد آلية عملها على استقبال معيارين حتميين بالتوالي: رابط الاتصال الفعال، ومتغير نص الاستعلام المجهز.

خطوات كتابة كود تنفيذ أمر الحفظ وإرساله للسيرفر: (mysqli_query())

1. حجز متغير لاستقبال نتيجة التنفيذ: تبدأ الجملة البرمجية بكتابة المتغير المسؤول عن حمل حالة نجاح أو فشل العملية متبوعاً بعلامة المساواة مثل \$result =

```
$result = mysqli_query($conn, $query);
```

2. استدعاء دالة التشغيل الفوري: ترك مسافة فارغة واحدة، ثم كتابة اسم الدالة القياسية بأحرف صغيرة `mysqli_query` متبوعة بفتح قوسين هلالين ().

3. تضمين رابط الاتصال الحاكم كمعيار أول: تمرير المتغير المخصص لحمل قنوات الربط المفتوحة والمستقرة متبوعاً بفاصلة عادية مثل `$conn`,

4. تمرير متغير أمر الحفظ المصاغ كمعيار ثانٍ: كتابة اسم المتغير النصي الذي يحمل سكربت الـ `INSERT INTO` (الذي تم تجهيزه في المهارة السابقة) كمعيار متمم مثل `$query`

5. إغلاق جملة الإرسال وإطلاق الأمر: غلق القوس الهلالي للدالة، ثم إنهاء السطر بالكامل بوضع الفاصلة المنقوطة؛ لتبدأ عملية الحقن والتخزين الفوري داخل السيرفر.

= متغير النتيجة \$

mysqli_query(متغير الاتصال, \$متغير الاستعلام);

```
$result =
```

```
mysqli_query($conn, $query);
```

بما أن هذا الاستعلام يقوم بإدخال بيانات جديدة، فإن المتغير `$result` سيحمل قيمة منطقية (Boolean). إذا تمت الإضافة بنجاح سيحمل القيمة `true`، وإذا فشلت سيحمل القيمة `false`. يُنصح دائماً باستغلال هذه النتيجة لطباعة رسالة تأكيد للمستخدم باستخدام جملة شرطية بسيطة:

```
if ($result) {  
    echo "تم حفظ البيانات بنجاح!";  
} else {  
    echo "حدث خطأ أثناء الحفظ."  
}
```


فحص صحة المدخلات وضمان تعبئة الحقول الإلزامية في النموذج البرمجي باستخدام دالتي (isset()) و (empty())

تُعد هذه المهارة الخطوة البرمجية الاستباقية لتأمين وتدقيق البيانات، حيث يتمثل دورها الوظيفي في التحقق من أن المستخدم قد ضغط بالفعل على زر الإرسال، وأن الحقول الإلزامية في نموذج الويب تحتوي على بيانات فعلية ولم تُترك فارغة. وتكمن الأهمية التعليمية والعملية لتطبيق دالتي (isset()) و (empty()) في منع السكريبت من معالجة قيم فارغة أو وهمية قد تؤدي لحدوث أخطاء برمجية داخل السيرفر؛ وتتعامل آلية عملها مع مصفوفات الاستقبال (مثل \$_POST) للتأكد من وجود المتغير وامتلائه بالبيانات قبل نقله لمرحلة الحفظ.

خطوات كتابة كود فحص وتأكد تعبئة الحقول الإلزامية:

1. صياغة الجملة الشرطية الاختبارية الأولى: تبدأ الجملة البرمجية بكتابة أداة الشرط if المتبوعة بفتح قوسين هلاليين واستدعاء دالة التحقق (isset()) للتأكد من إرسال النموذج (مثل if (isset(\$_POST['save_btn']))).

```
if (isset($_POST['save_btn']) && !empty($_POST['student_name'])) {  
    // إذا تحقق الشرطان: زر الإرسال نُقر، والحقل ليس فارغاً  
    $user_name = $_POST['student_name'];  
} else {  
    // إذا لم يتحقق أحد الشرطين أو كلاهما  
    echo "الرجاء تعبئة حقل الاسم الإلزامي!"  
}
```

2. دمج فحص الحقول الفارغة بنفي الدالة: وضع معامل الربط المنطقي && ثم دمج أداة النفي !مع دالة الفحص empty() للتأكد من أن الحقل ليس فارغاً (مثل && !empty(\$_POST['student_name'])).

3. حجز المتغير واستقبال القيمة بأمان: فتح قوس مجعد {والانتقال لسطر جديد لحجز المتغير البرمي واستقبال القيمة المدخلة بداخل الخادم (مثل \$user_name = \$_POST['student_name'];).

4. صياغة رسالة الخطأ البديلة :الانتقال لسطر جديد وإغلاق القوس المجعد، ثم كتابة مسار الفشل
else وطباعة رسالة تنبيهية للمستخدم في حال ترك الحقل فارغاً.
5. إنهاء المعاملة الشرطية بالكامل :غلق القوس المجعد النهائي لكتلة الشرط ووضع الفاصلة المنقوطة
; لإنهاء الأوامر الداخلية للغة PHP بدقة.

```
if (isset($_POST['الإرسال']) && !empty($_POST['الاسم حقل النموذج']))  
  
{  
  
    $الاسم حقل النموذج = متغير القيمة;  
  
} else {  
  
    echo "رسالة التنبيه";  
  
}
```

```
if (isset($_POST['save_btn']) && !empty($_POST['student_name']))  
  
{  
  
    $user_name = $_POST['student_name'];  
  
}  
  
else {  
  
    echo "عذراً، يجب ملء حقل اسم الطالب أولاً";  
  
}
```

اختبار منطق البيانات ومطابقتها للمعايير المطلوبة وشروط النظام قبل الحفظ باستخدام جملة (if...else)

تُعد هذه المهارة الأداة البرمجية الحاكمة لتطبيق قواعد العمل (Business Logic) ، حيث يتمثل دورها الوظيفي في إخضاع البيانات المدخلة والمخزنة مؤقتاً في المتغيرات لسلسلة من الاختبارات الرياضية أو المنطقية قبل السماح بمرورها للجدول الفعلي. وتكمن الأهمية التعليمية والعملية لتطبيق جملة if...else في هذا السياق في تدريب الطالب على منع تسجيل بيانات غير منطقية أو مخالفة لشروط النظام التعليمي (مثل رفض حفظ درجة الطالب إذا كانت أقل من صفر أو أكبر من 100، أو رفض تسجيل كلمة مرور قصيرة)؛ وتعتمد آلية عملها على توجيه المعالج لتمرير السكربت فقط في حال مطابقة القيمة للمعيار الصارم المحدد.

خطوات كتابة كود اختبار منطق البيانات وشروط النظام:

1. صياغة الجملة الشرطية للمعيار الحاكم: تبدأ الجملة البرمجية بكتابة أداة الشرط if متبوعة بقوسين هلاليين يكتب بداخلها المتغير ومعامل المقارنة الحاكم (مثل if (\$grade_score >= 0 && \$grade_score <= 100)).

```
if ($grade_score >= 0 && $grade_score <= 100) {  
    } else {  
        echo "الدرجة غير صالحة، يجب أن تكون بين 0 و 100";  
    }
```

2. فتح كتلة الموافقة البرمجية: فتح قوس مجعد {والانتقال لسطر جديد لكتابة كود الاستمرار والتجهيز لعملية الحفظ الآمنة عند نجاح الاختبار.

3. صياغة مسار الرفض والتنبيه البدي: الانتقال لسطر جديد وإغلاق القوس المجعد وكتابة العبارة else متبوعة بفتح قوس مجعد جديد لإعلان فشل المطابقة.

4. طباعة الوصف النصي للمشكلة: الانتقال لسطر جديد واستدعاء أمر الطباعة echo لعرض رسالة توضح المعيار الصحيح للنظام (مثل طباعة "الدرجة غير صالحة").

5. إغلاق المسارات الشرطية بالكامل: غلق القوس المجعد لكتلة الرفض والتأكد من قفل كافة الأوامر والعبارات الداخلية بالفاصلة المنقوطة; .

```

{ (متغير القيمة المعيار الأول && $متغير القيمة المعيار الثاني $) if

    كود التجهيز للحفظ هنا //

} else {

    echo "رسالة الرفض المنطقي ";

}

```

```

if ($grade_score >= 0 && $grade_score <= 100) {

    كود التجهيز للحفظ في القاعدة سيكتب هنا لضمان منطقية الدرجة //

} else {

    echo "خطأ: يجب أن تكون درجة الطالب بين 0 و 100 فقط";

}

```

بناء استعلام الإضافة البرمجي لنقل البيانات المدخلة وتخزينها في الجداول المستهدفة باستخدام أمر
(INSERT INTO)

تُعد هذه المهارة الخطوة الهندسية التأسيسية لتشكيل جملة التغذية الديناميكية، حيث يتمثل دورها الوظيفي في صياغة سكريبت SQL الخاص بعملية الإدخال وحفظه بالكامل داخل متغير نصي في PHP ، ليربط بدقة متناهية بين الأعمدة الفيزيائية للجدول والمتغيرات البرمجية التي تحمل قيم المدخلات المفحوصة والمؤمنة. وتكمن الأهمية التعليمية والعملية لتطبيق هذا البناء في تدريب الطالب على إعداد حزم البيانات وتجهيز السكريبتات بأسلوب مرن وقابل للتغير تلقائياً مع كل عملية تسجيل جديدة دون تعديل جوهر الكود؛ وتعتمد آلية عملها على حصر النص البرمي لل SQL بين علامات تنصيب مزدوجة مع وضع المتغيرات داخل علامات تنصيب مفردة.

خطوات بناء وصياغة استعلام الإضافة البرمجي:

1. **حجز المتغير النصي للاستعلام:** تبدأ الجملة البرمجية بكتابة المتغير المخصص لحمل السكريبت بالكامل متبوعاً بمعامل المساواة مثل `$query =`.

```
$query =
```

2. **فتح الحاضنة النصية للأمر:** وضع علامتي تنصيص مزدوجة `"` للإعلان عن بدء كتابة لغة قواعد البيانات SQL من داخل كود الـ PHP.
3. **كتابة كود الإدخال وتوصيف الأعمدة:** كتابة العبارة المحجوزة بأحرف كبيرة `INSERT INTO` متبوعة باسم الجدول، ثم فتح قوسين هلاليين لتحديد الأعمدة المستهدفة بالتتابع (مثل `INSERT INTO students (student_name, grade)`).

```
$query = "INSERT INTO students (student_name, grade)
```

4. **حقن وربط متغيرات القيم بالتوالي:** كتابة الكلمة المفتاحية `VALUES` متبوعة بفتح قوسين ووضع المتغيرات البرمجية الحاضنة للقيم بالترتيب بين علامات تنصيص مفردة وفصلها بفاصلة (مثل `VALUES ('$user_name', '$grade_score')`).

```
$query = "INSERT INTO students (student_name, grade) VALUES ('$user_name', '$grade_score')";
```

5. **إغلاق السكريبت الهيكلي المدمج:** وضع الفاصلة المنقوطة الداخلية لـ SQL ثم غلق علامة التنصيص المزدوجة، وإنهاء السطر بالكامل بوضع الفاصلة المنقوطة للغة.;

= متغير الاستعلام \$

```
$query =
```

```
"INSERT INTO students (student_name, grade) VALUES  
('$user_name', '$grade_score');"
```

تنفيذ الاستعلام ويرسله للسيرفر، ثم يوجه المتصفح تلقائياً لصفحة أخرى بعد نجاح العملية باستخدام دالتي `header()` و `mysqli_query()`

تُعد هذه المهارة الأداة التنفيذية والتحويلية الختامية لدورة حياة معالجة البيانات، حيث يتمثل دورها الوظيفي في إطلاق وإرسال أمر الحفظ المجهز إلى خادم MySQL لتسجيله، وفور نجاح العملية يتم إصدار أمر فوري للمتصفح لمغادرة الصفحة الحالية والانتقال تلقائياً إلى صفحة أخرى (مثل صفحة عرض الجدول أو لوحة التحكم). وتكمن الأهمية التعليمية والعملية لتطبيق دالتي `mysqli_query()` و `header()` معاً في تحسين تجربة المستخدم وعزل العمليات؛ حيث يمنع توجيه المتصفح إعادة إرسال البيانات بالخطأ عند تحديث الصفحة (Duplicate Submission)؛ وتعتمد آلية عملها على فحص نتيجة دالة التنفيذ وتمرير المسار الجغرافي الجديد عبر الدالة `Location`.

خطوات كتابة كود تنفيذ الاستعلام وتوجيه المتصفح تلقائياً:

1. إرسال وتنفيذ أمر الحفظ: تبدأ الجملة البرمجية بحجز متغير النتيجة متبوعاً بدالة التنفيذ القياسية بأحرف صغيرة وتمرير رابط الاتصال ومتغير الاستعلام (مثل `$result = mysqli_query($conn, $query);`).

```
1  $result = mysqli_query($conn, $query);
2  if ($result) {
3      header("Location: view_students.php");
4  }
```

2. فحص نجاح عملية التخزين: الانتقال لسطر جديد وبناء جملة شرطية `if` لفحص تحقق متغير النتيجة وتأكد استقبال السيرفر للأمر (مثل `if ($result)`).

3. استدعاء دالة التوجيه التلقائي: فتح قوس مجعد `{` والانتقال لسطر جديد وكتابة دالة التحويل القياسية بأحرف صغيرة `header` متبوعة بفتح قوسين هلاليين `()`.

4. تحديد وجهة الصفحة الجديدة بدقة: وضع علامتي تنصيص مزدوجة وتضمين الكلمة المفتاحية الحاكمة `Location` متبوعة مباشرة بالاسم الصريح للامتداد والصفحة المستهدفة (مثل `"Location: view_students.php"`).

5. إغلاق الأوامر وقفل المسار : غلق القوس الهلالي لدالة التوجيه وإنهاء السطر بالفاصلة المنقوطة ;
ثم النزول لسطر جديد وإغلاق القوس المجدد {} لإتمام حزمة السكريبت الآمنة.

```
(متغير الاتصال, $متغير الاستعلام) = mysqli_query($
```

```
if (متغير النتيجة) {
```

```
    echo "رسالة النجاح المباشرة";
```

```
}
```

```
$result = mysqli_query($conn, $query);
```

```
if ($result) {
```

```
    echo "تم تنفيذ الاستعلام بنجاح";
```

```
}
```

استرجاع كلمات وفئات البحث المرسله وتخزينها برمجياً في الذاكرة باستخدام مصفوفة الإرسال الظاهرة (\$_GET)

تُعد هذه المهارة الأداة البرمجية التأسيسية لبناء محركات البحث الداخلية، حيث يتمثل دورها الوظيفي في النقاط البيانات أو الكلمات المفتاحية التي كتبها الزائر في خانة البحث وضغط لإرسالها، ليقوم السكريبت بسحبها من شريط العنوان وتخزينها في متغيرات مستقلة. وتكمن الأهمية التعليمية والعملية لتطبيق مصفوفة \$_GET في تمكين الطالب من بناء أنظمة بحث مرنة تتيح للمستخدمين مشاركة رابط نتائج البحث مع الآخرين أو الرجوع إليه (مثل الاحتفاظ بكلمة البحث وفئة التصفية ظاهرة في الرابط)؛ وتعتمد آلية عملها على قراءة المفاتيح الممررة عبر الرابط وتفرغ محتواها النصي فوراً داخل ذاكرة السيرفر مؤقتاً.

خطوات كتابة كود استرجاع وتخزين كلمات وفئات البحث: (\$_GET)

1. فحص إرسال طلب البحث علناً: تبدأ الجملة البرمجية بكتابة أداة الشرط if متبوعة باستدعاء دالة

الفحص isset() للتأكد من قيام المستخدم بالضغط على زر البحث مثل if
(isset(\$_GET['search_btn']))

```
1  if (isset($_GET['search_btn'])) {  
2      // استقبال كلمة البحث  
3      $search_word = $_GET['keyword'];  
4  
5      // استقبال فئة البحث المحددة  
6      $filter_cat = $_GET['category'];  
7  }
```

2. فتح الحاضنة البرمجية للمتغيرات: فتح قوس مجعد {والانتقال لسطر جديد للبدء في حجز واستقبال القيم الممررة.

3. استرجاع وتخزين كلمة البحث: كتابة رمز متغير الكلمة متبوعاً بمعامل المساواة ومصفوفة الاستقبال

لجلب ما كُتب في حقل النص مثل \$search_word = \$_GET['keyword'];

4. استرجاع وتخزين فئة التصنيف: الانتقال لسطر جديد وكتابة رمز متغير الفئة متبوعاً بمصفوفة

الاستقبال لجلب الخيار المنتقى من القائمة المنسدلة مثل \$filter_cat = \$_GET['category'];

5. إغلاق كتلة الاستقبال الشرطية: الانتقال لسطر جديد وإغلاق القوس المجعد للشرط {مع التأكد من إنهاء السطور البرمجية الداخلية بالفاصلة المنقوطة ;.}

```
if (isset($_GET['زر البحث'])) {
```

```
    $اسم حقل الكلمة = $_GET['كلمة البحث'];
```

```
    $اسم قائمة الفئة = $_GET['متغير فئة البحث'];
```

```
}
```

```
if (isset($_GET['search_btn'])) {
```

```
    $search_word = $_GET['keyword'];
```

```
    $filter_cat = $_GET['category'];
```

```
}
```


معالجة نصوص الاستعلام المدخلة لتأمينها وتنقية مخرجات البحث من هجمات الاختراق باستخدام دالة (mysql_real_escape_string())

تُعد هذه المهارة الركيزة الأمنية الحتمية لحماية محركات البحث، حيث يتمثل دورها الوظيفي في تنظيف العبارات والكلمات التي يدخلها المستخدمون في حقل البحث وتطهيرها من أي رموز خبيثة قبل تمريرها للاستعلام الفعلي. وتكمن الأهمية التعليمية والعملية لتطبيق دالة (mysql_real_escape_string()) في هذا السياق في حظر هجمات الـ (SQL Injection)؛ حيث يستهدف المخترقون ثغرات حقول البحث بكتابة أكواد برمجية بدلاً من كلمات البحث العادية لتدمير قاعدة البيانات أو تسريب السجلات؛ وتعتمد آلية عملها على إبطال مفعول الرموز الخاصة وتحويلها لنصوص عادية آمنة تماماً.

خطوات كتابة كود تأمين وتنقية نصوص البحث: (mysql_real_escape_string())

1. **حجز المتغير الحاضر للنص المؤمن:** تبدأ الجملة البرمجية بكتابة رمز المتغير الجديد والمخصص لحفظ الكلمة بعد التطهير مثل `$secure_word =`
2. **استدعاء دالة التنظيف القياسية:** تترك مسافة فارغة واحدة، ثم كتابة اسم الدالة بأحرف صغيرة `mysql_real_escape_string` متبوعة بفتح قوسين هلاليين () .
3. **تضمين رابط الاتصال الحاكم كمتغير أول:** تمرير المتغير الخاص بقناة الاتصال المفتوحة والفعالة متبوعاً بفاصلة عادية مثل `$conn,`
4. **تمرير متغير كلمة البحث الأصلي كمتغير ثانٍ:** كتابة اسم المتغير الذي يحمل الكلمة الخام القادمة من مصفوفة الاستقبال لتنقيتها مثل `$search_word`

```
$secure_word = mysql_real_escape_string($conn, $search_word);
```

5. **إغلاق جملة التنظيف وقفل السطر:** غلق القوس الهاللي جهة اليمين، ثم وضع الفاصلة المنقوطة ; للإشارة إلى اكتمال مسار الحماية وجاهزية المتغير الآمن للاستخدام في الفترة.

= متغير النص المؤمن \$

mysql_real_escape_string(\$متغير الاتصال, \$متغير كلمة البحث الأصلي);

\$secure_word =

mysql_real_escape_string(\$conn, \$search_word);

بناء استعلام البحث المطور عن جزء من النص أو الكلمات الدلالية داخل قاعدة البيانات باستخدام معامل المطابقة (LIKE)

تُعد هذه المهارة الأداة البرمجية المرنة لجلب النتائج التقريبية والذكية، حيث يتمثل دورها الوظيفي في صياغة جملة استعلام SQL قادرة على فحص الأعمدة والعثور على السجلات التي تحتوي على الكلمة المطلوبة أو جزء منها، دون اشتراط التطابق الكامل للحروف. وتكمن الأهمية التعليمية والعملية لتطبيق معامل المطابقة LIKE في تزويد الطالب بمهارة بناء محركات بحث احترافية وسهلة الاستخدام (مثل البحث عن اسم طالب بكتابة اسمه الأول فقط، أو البحث عن مادة بكتابة كلمة دلالية من مسمّاها)؛ وتعتمد آلية عملها على دمج علامات النسبة المئوية (%) حول المتغير المؤمن لتعني البحث عن الكلمة في أي مكان داخل الحقل النصي.

خطوات بناء وصياغة استعلام البحث بمعامل المطابقة: (LIKE)

1. **حجز متغير لحفظ الاستعلام النصي**: تبدأ الجملة البرمجية بكتابة المتغير المخصص لحمل السكريبت بالكامل متبوعاً بعلامة المساواة مثل `$query =`
2. **فتح علامات التنصيص الحاضرة للأمر**: وضع علامتي تنصيص مزدوجة `"` لبدء صياغة لغة الـ SQL من داخل صفحة الـ PHP .
3. **كتابة أمر الاسترجاع القياسي وتحديد الجدول**: كتابة العبارة المحجوزة `SELECT * FROM` متبوعة بالاسم الصريح للجدول المستهدف مثل `SELECT * FROM students`
4. **توظيف قيد التصفية ومعامل المطابقة**: كتابة أداة الشرط `WHERE` يليه اسم العمود المراد فحص محتواه ثم المعامل `LIKE` بأحرف كبيرة مثل `WHERE student_name LIKE`
5. **حقن المتغير المؤمن بعلامات النسبة المئوية**: وضع علامتي تنصيص مفردة وبداخلهما رمز النسبة المئوية محيطاً بمتغير النص المؤمن، ثم غلق علامة التنصيص المزدوجة وإنهاء السطر بالكامل بالفاصلة المنقوطة للغة ; PHP مثل `'%$secure_word%'`;

```
$query = "SELECT * FROM students WHERE student_name LIKE '%$secure_word%'";
```

= متغير الاستعلام \$

```
"%متغير النص المؤمن%" LIKE اسم العمود WHERE اسم الجدول "SELECT * FROM
```

```
$query =
```

```
"SELECT * FROM students WHERE student_name LIKE
```

```
'%$secure_word%';
```

صياغة جمل الاستعلام المركبة ويضيف شروط التصفية ديناميكياً بناءً على خيارات المستخدم باستخدام معامل الربط النصي(.=).

تُعد هذه المهارة الأسلوب البرمجي الذكي لبناء محركات البحث المتقدمة ومتعددة الخيارات (Advanced Search)، حيث يتمثل دورها الوظيفي في دمج وإضافة شروط تصفية إضافية إلى جملة الاستعلام الأساسية بشكل تدريجي وتلقائي فقط إذا قام المستخدم باختيار فئات محددة. وتكمن الأهمية التعليمية والعملية لتطبيق معامل الربط النصي =. في جعل السكربت يتكيف ديناميكياً مع رغبة الزائر (مثل تضيق نطاق البحث لجلب الطلاب الذين يحتوي اسمهم على الكلمة المطلوبة "وفي نفس الوقت" ينتمون لقسم تكنولوجيا التعليم تلميساً لخياره)؛ وتعتمد آلية عملها على التحقق من امتلاء متغير الفئات ثم استخدام معامل الدمج لربط عبارة AND بنهاية نص الاستعلام دون مسح الجزء التأسيسي السابق.

خطوات صياغة جمل الاستعلام المركبة بمعامل الربط النصي: (.=).

1. تجهيز جملة الاستعلام التأسيسية: تبدأ الجملة البرمجية بتهيئة المتغير وصياغة أمر البحث الأساسي

الحتمي وحفظه مثل: `$query = "SELECT * FROM students WHERE 1=1";`

2. بناء الشرط الاختباري للفئة الإضافية: الانتقال لسطر جديد وبناء جملة الشرط if للتأكد من أن متغير

فئة البحث ليس فارغاً مثل `if (!empty($filter_cat))`

```
$query = "SELECT * FROM students WHERE 1=1";  
if (!empty($filter_cat)) {  
    $query .= " AND department = '$filter_cat'";  
}
```

3. فتح حاوية دمج شروط التصفية: فتح قوس مجعد {والانتقال لسطر جديد لبدء عملية اللصق والربط النصي الديناميكي.

4. توظيف معامل الربط لإضافة العبارة الشرطية: كتابة اسم متغير الاستعلام متبوعاً مباشرة بمعامل الدمج اللغوي واللصق =. ثم علامات تنصيص مزدوجة يكتب بداخلها شرط التصفية الإضافي مسبقاً بمسافة (مثل). \$query .= " AND department = '\$filter_cat'";

5. إغلاق كتلة الدمج الديناميكية بالكامل: الانتقال لسطر جديد وإغلاق القوس المجعد للشرط {} والتأكد من قفل الأسطر الداخلية بالفاصلة المنقوطة للغة PHP ;.

```
$query = "SELECT * FROM اسم الجدول WHERE 1=1";
```

```
if (!empty($filter_cat)) {
```

```
    $query .= " AND department = '$filter_cat'";
```

```
}
```

```
$query = "SELECT * FROM students WHERE 1=1";
```

```
if (!empty($filter_cat)) {
```

```
    $query .= " AND department = '$filter_cat'";
```

```
}
```

عرض سجلات النتائج المفطرة ويحدث محتوى جدول البيانات النهائي أمام المستخدم فور تنفيذ استعلام البحث

تعد هذه المهارة المحطة الختامية والدور التنفيذي المباشر لعرض ثمار عملية الفرز، حيث يتمثل دورها الوظيفي في ضخ أمر البحث المركب والمؤمن إلى السيرفر، وتدوير السجلات المسترجعة لتحديث محتوى جدول الـ HTML أمام المستخدم بالنتائج الجديدة فوراً. وتكمن الأهمية التعليمية والعملية لتطبيق هذه المهارة

في تمكين الطالب من ربط المخرجات بقوالب العرض واجهات الويب؛ بحيث يختفي الجدول الشامل ويحل محله تلقائياً الصفوف المفلتره التي تطابق معايير بحث الزائر بدقة؛ وتعتمد آلية عملها على تمرير المتغير المركب عبر الدالة `mysqli_query()` ثم تفكيك الصفوف بواسطة دوال الجلب التكرارية (مثل `while`).

خطوات تنفيذ استعلام البحث وعرض السجلات المفلتره:

1. إرسال وتشغيل أمر البحث المركب: تبدأ الجملة البرمجية بحجز متغير النتيجة متبوعاً بدالة التنفيذ القياسية بأحرف صغيرة وتمرير رابط الاتصال ومتغير الاستعلام المركب مثل `$result = mysqli_query($conn, $query);`

```
// تشغيل الاستعلام وجلب النتيجة من السيرفر.1
$result = mysqli_query($conn, $query);

// بدء حلقة التكرار لتفكيك الصفوف صفاً تلو الآخر.2
while ($row = mysqli_fetch_assoc($result)) {

    // طباعة البيانات وعرضها ديناميكياً داخل وسوم الجدول.4 و 3
    echo "<tr><td>" . $row['student_name'] . "</td></tr>";

} // إغلاق حلقة التكرار وتأمين نهاية السكريبت.5
```

2. بناء حلقة التدوير لاستخراج الصفوف المفلتره: الانتقال لسطر جديد وكتابة أمر التكرار `while` متبوعاً بفتح قوسين هلالين لتفكيك السجلات العائدة وتوليدها صفاً تلو الآخر (مثل `$row = mysqli_fetch_assoc($result)`).

3. فتح كتلة العرض داخل الجدول: فتح قوس مجعد {والانتقال لسطر جديد للبدء في طباعة خلايا الجدول الحاضنة للبيانات الجديدة.

4. طباعة وعرض الحقول المحدثة أمام المستخدم: استدعاء أمر الطباعة `echo` وتمرير وسوم الجدول مدمجاً بداخلها مصفوفة الصف لاستخراج القيمة الصريحة وتحديث الواجهة (مثل `echo $row['student_name'];`).

5. إغلاق حلقة التكرار البرمجية: إنهاء السكريبت بالنزول لسطر جديد وإغلاق القوس المجعد للحلقة {} مع التأكد من قفل العبارات البرمجية بالفاصلة المنقوطة ;.

```

(متغير الاتصال, $متغير الاستعلام) = mysqli_query(
    متغير النتيجة = mysqli_fetch_assoc($متغير النتيجة) {
        متغير الصف['اسم العمود'];
    }

```

```

$result = mysqli_query($conn, $query);

while ($row = mysqli_fetch_assoc($result)) {
    echo "اسم الطالب " . $row['student_name'] . "<br>";
}

```

مهارات تعديل وتحديث بيانات المستخدمين (Data Editing Skills).

استخلاص المعرف الفريد للسجل ID المطلوب تعديله من رابط الصفحة باستخدام مصفوفة الإرسال الظاهرة (\$_GET)

تعد هذه المهارة الخطوة التأسيسية لعمليات التحديث الديناميكي، حيث يتمثل دورها الوظيفي في التقاط الرقم الكودي الفريد (ID) للمستخدم أو الطالب المراد تعديل بياناته، والممرر علناً عبر رابط الصفحة (URL) فور ضغط المعلم أو المسؤول على زر التعديل في صفحة العرض. وتكمن الأهمية التعليمية والعملية لتطبيق مصفوفة \$_GET في هذا السياق في جعل صفحة التعديل ذكية ومخصصة؛ بحيث تفتح لقراءة سجل شخص واحد بعينه دون بقية السجلات؛ وتعتمد آلية عملها على قراءة المفتاح الرقمي الملحق بالرابط وتخزينه فوراً في متغير مستقل داخل الذاكرة لتقييد الاستعلامات اللاحقة به.

خطوات كتابة كود استخلاص المعرف الفريد من الرابط: (\$_GET)

1. فحص وجود المعرف في الرابط: تبدأ الجملة البرمجية بكتابة أداة الشرط `if` امتبوعة باستدعاء دالة

الفحص `isset()` للتأكد من تمرير الرقم الكودي عبر الرابط مثل `if (isset($_GET['id']))`

```
if (isset($_GET['id'])) {  
    $user_id = $_GET['id'];  
}
```

2. فتح حاضنة الاستقبال البرمجية: فتح قوس مجعد `{` والانتقال لسطر جديد للبدء في سحب القيمة الرقمية وحفظها.

3. حجز المتغير واستخلاص قيمة المعرف: كتابة رمز المتغير المخصص لحمل الكود متبوعاً بمعامل المساواة ومصفوفة الاستقبال لجلب الرقم مثل `$user_id = $_GET['id'];`

4. إغلاق كتلة الاستقبال الشرطية: الانتقال لسطر جديد وإغلاق القوس المجعد للشرط `{}` لإنهاء مسار التحقق بأمان.

5. إنهاء الأسطر بالفاصلة المنقوطة: التأكد من وضع الفاصلة المنقوطة في نهاية أمر الاستخلاص الداخلي لمطابقة القواعد النحوية للغة.

```
if (isset($_GET['الرابط في الرابط'])) {
```

```
    $اسم المعرف في الرابط = $_GET['الرابط في الرابط'];
```

```
}
```

```
if (isset($_GET['id'])) {
```

```
    $user_id = $_GET['id'];
```

```
}
```

استرجاع بيانات السجل الحالي من القاعدة ويعرضها مسبقاً داخل حقول نموذج التعديل باستخدام استعلام (SELECT)

تُعد هذه المهارة الأداة البرمجية لتحسين تجربة المستخدم وعرض البيانات القائمة-Data Pre (filling)، حيث يتمثل دورها الوظيفي في جلب الصف الخاص بالمعرّف المستخلص من قاعدة البيانات وتفكيكه وضخ قيمه القديمة مسبقاً داخل خانات نموذج الـ HTML. وتكمن الأهمية التعليمية والعملية لتطبيق استعلام الـ SELECT المقيد في تعريف الطالب بكيفية تذكر المستخدم بالبيانات الحالية قبل تعديلها (مثل ظهور اسم الطالب الحالي "خالد" داخل الخانة تلقائياً ليقوم المستخدم بتعديله إلى "خالد محمود" بدلاً من الكتابة من الصفر)؛ وتعتمد آلية عملها على دمج دالة `mysqli_query()` مع دالة الجلب `mysqli_fetch_assoc()`.

خطوات كتابة كود استرجاع وعرض البيانات الحالية داخل النموذج:

1. صياغة استعلام الجلب المقيد بالمعرّف: تبدأ الجملة البرمجية بحجز متغير الاستعلام وصياغة أمر SQL لجلب السجل المطابق للرقم الكودي (مثل `$query = "SELECT * FROM students WHERE student_id = $user_id"`).

```
<?php
// جلب البيانات من السيرفر وتجهيزها 3 و 2 و 1 :
$query = "SELECT * FROM students WHERE student_id = $user_id";
$result = mysqli_query($conn, $query);
$row = mysqli_fetch_assoc($result);
?>
```

2. تنفيذ الاستعلام وحفظ حزمة مخرجاته: الانتقال لسطر جديد وتمرير السكربت عبر دالة التنفيذ وتخزين النتيجة (مثل `$result = mysqli_query($conn, $query);`).

3. تفكيك وتحويل السجل إلى مصفوفة بيانات: الانتقال لسطر جديد واستدعاء دالة الجلب المصفوفي بأحرف صغيرة لحفظ الحقول (مثل `$row = mysqli_fetch_assoc($result);`).

4. استخراج القيمة وعرضها داخل وسم الإدخال: الانتقال إلى وسم الـ HTML الخاص بحقل الإدخال، وتوظيف الخاصية `value` وطباعة الحقل بداخلها (مثل `value="<?php echo $row['student_name']; ?>"`).

```
<form action="update.php" method="POST">
    <label>اسم الطالب</label>
    <input type="text" name="student_name" value="<?php echo $row['student_name']; ?>">

    <button type="submit" name="update_btn">حفظ التعديلات</button>
</form>
```


5. قفل جملة الطباعة والأوامر البرمجية: التأكد من إغلاق أسطر الـ PHP المنبثقة بالفاصلة المنقوطة
لضمان ظهور النص القديم مستقراً داخل النموذج.

```
"اسم حقل المعرف = $متغير_رقم_الكود WHERE اسم الجدول * FROM $متغير_الاستعلام
```

```
متغير الاتصال, $متغير_الاستعلام);
```

```
متغير النتيجة);
```

```
$query = "SELECT * FROM students WHERE student_id = $user_id";
```

```
$result = mysqli_query($conn, $query);
```

```
$row = mysqli_fetch_assoc($result);
```

توظيف حقل الإدخال المخفي Hidden Field لتخزين وتمرير معرف السجل ID بأمان دون إظهاره
للمستخدم أثناء الإرسال

تُعد هذه المهارة الحيلة الهندسية الأهم للحفاظ على سياق المعاملة البرمجية (State Management)، حيث يتمثل دورها الوظيفي في زرع حقل إدخال غير مرئي بصرياً على الشاشة داخل نموذج الـ HTML يحمل قيمة الرقم الكودي ID للسجل المفتوح، ليتم إرساله مع البيانات الجديدة فور الضغط على زر التحديث. وتكمن الأهمية التعليمية والعملية لتطبيق الحقل المخفي Hidden Field في تمكين سكربت الواجهة الخلفية من معرفة "أي صف بالظبط سيتم تحديثه" عند استقبال الطلب؛ لأن رابط الصفحة الذي يحتوي على الـ ID اختفي تأثيره بمجرد ضغط زر الإرسال عبر مصفوفة POST_؛ وتعتمد آلية عملها على نوع الحقل. type="hidden".

خطوات كتابة وتوظيف حقل الإدخال المخفي (Hidden Field)

1. فتح وسم الإدخال داخل النموذج الفعال: تبدأ الخطوة بكتابة وسم الإدخال القياسي لـ HTML في أي مكان بين وسمي البداية والنهاية للنموذج. (<input)

```
<form action="update_script.php" method="POST">
```

2. تحديد نوع الحقل البرمجي غير المرئي: ترك مسافة فارغة وإسناد القيمة المخفية لخاصية النوع لمنع ظهوره للمستخدم مثل type="hidden"

```
<form action="update_script.php" method="POST">  
  <input type="hidden" name="student_id" value="<?php echo $row['student_id']; ?>">
```

3. تسمية الحقل لغايات الاستقبال الخلفي: إسناد اسم برمجي صريح للخاصية name ليتم التقاطه عبر مصفوفة الإرسال لاحقاً مثل name="student_id"

4. حقن قيمة المعرف الحالي برمجياً: توظيف الخاصية value وفتح سكربت PHP مصغر لطباعة رقم الكود المخزن في مصفوفة الجلب مثل value="<?php echo \$row['student_id']; ?>"

```
<form action="update_script.php" method="POST">  
  <input type="hidden" name="student_id" value="<?php echo $row['student_id']; ?>">  
  <label>اسم الطالب</label>  
  <input type="text" name="student_name" value="<?php echo $row['student_name']; ?>">  
  <button type="submit" name="update_btn">تحديث البيانات</button>  
</form>
```

5. إغلاق وسم الحقل المخفي بدقة: إنهاء السطر بغلاق وسم الإدخال > والتأكد من قفل أمر الطباعة بالفاصلة المنقوطة ; لضمان تمرير الكود خفية خلف الكواليس.

```
<input type="hidden" name="اسم الحقل المخفي" value="<?php echo $['اسم حقل المعرف']; ?>">
```

```
<input type="hidden" name="student_id" value="<?php echo $row['student_id']; ?>">
```

بناء استعلام التحديث البرمجي لاستبدال القيم القديمة بالبيانات الجديدة المعدلة في الجدول باستخدام أمر (UPDATE)

تُعد هذه المهارة المحور البرمجي لإعادة صياغة البيانات، حيث يتمثل دورها الوظيفي في تشكيل سكربت SQL الخاص بعملية التعديل وحفظه كاملاً داخل متغير نصي في PHP، ليربط بين الأعمدة الفيزيائية للجدول والقيم الجديدة المستقاة من النموذج، مع تقييد العملية بالمعرف الفريد الممرر عبر الحقل المخفي. وتكمن الأهمية التعليمية والعملية لتطبيق هذا البناء في تدريب الطالب على صياغة أوامر صيانة وتعديل المحتوى بدقة وعناية حظراً لإتلاف بقية بيانات الجدول؛ وتعتمد آلية عملها على توظيف قيد الشرط الصارم WHERE للتأكد من أن التحديث سيصيب السجل المستهدف حصرياً.

خطوات بناء وصياغة استعلام التحديث البرمجي: (UPDATE)

1. التقاط القيم المستجدة والمعرف المخفي: تبدأ الجملة البرمجية بسحب البيانات الحالية والمعرف من

مصفوفة الإرسال مثل \$user_id = \$_POST['student_id']; و \$new_name = \$_POST['student_name'];

```
$user_id = $_POST['student_id'];  
$new_name = $_POST['student_name'];  
$query = "UPDATE students SET student_name = '$new_name' WHERE student_id = $user_id";  
$result = mysqli_query($conn, $query);
```

2. حجز المتغير النصي للاستعلام: كتابة المتغير المسؤول عن حمل السكريبت متبوعاً بمعامل المساواة

وفتح علامتي تنصيص مزدوجة مثل \$query = ""

3. صياغة أمر التعديل وتحديد الجدول المستهدف: كتابة العبارة المحجوزة بأحرف كبيرة UPDATE

متبوعة باسم الجدول، ثم كلمة التعيين SET مثل UPDATE students SET

4. توصيف الأعمدة وتمرير القيم الجديدة: كتابة اسم الحقل متبوعاً بمعامل المساواة ثم المتغير الجديد

بين علامات تنصيص مفردة مثل student_name = '\$new_name'

5. تقييد المعاملة بالشرط الحاكم وقفل السطر: كتابة أداة الشرط WHERE متبوعة بمساواة المفتاح

الرئيسي بالمعرف الممرر خفية، ثم غلق علامتي التنصيص المزدوجة وإنهاء السطر بالفاصلة المنقوطة

لغة PHP؛ مثل WHERE student_id = \$user_id";

"اسم حقل المعرف = \$متغير الرقم الكودي WHERE 'اسم العمود = '\$متغير القيمة الجديدة SET اسم الجدول UPDATE" = متغير الاستعلام

```
$query = "UPDATE students SET student_name = '$new_name' WHERE student_id = $user_id";
```

نسيان جملة WHERE في استعلام التحديث هو أحد أشهر الكوابيس البرمجية! إذا كتبت الأمر هكذا :
`UPDATE students SET student_name = '$new_name'`
سيقوم السيرفر بتغيير أسماء جميع الطلاب في قاعدة البيانات إلى هذا الاسم الجديد! لذلك، شرط التقييد بالمعرف (ID) هو صمام الأمان الإلزامي لحماية البيانات.

تنفيذ أمر التحديث على السيرفر، ثم يوجه المتصفح تلقائياً إلى صفحة عرض البيانات بعد النجاح باستخدام دالتي `(mysql_query())` و `(header())`

تُعد هذه المهارة الأداة التنفيذية والتحويلية الختامية لدورة حياة تعديل البيانات، حيث يتمثل دورها الوظيفي في ضخ أمر التحديث المجهز والمقيد إلى خادم MySQL لتعديل الصف، وفور التثبيت الناجح يتم إرسال أمر فوري للمتصفح بمغادرة صفحة التعديل والعودة تلقائياً لصفحة العرض الشاملة لرؤية البيانات في شكلها المستجد. وتكمن الأهمية التعليمية والعملية لتطبيق دالتي `(mysql_query())` و `(header())` معاً في عزل العمليات البرمجية وتقديم واجهة ذكية ومناسبة للمستخدم تضمن إعلامه باكمال التعديل عملياً؛ وتعتمد آلية عملها على فحص حالة متغير النتيجة المنطقي لإطلاق التوجيه عبر العبارة `Location.`

خطوات كتابة كود تنفيذ أمر التحديث وتوجيه المتصفح تلقائياً:

1. إرسال وتشغيل أمر التعديل المقيد: تبدأ الجملة البرمجية بحجز متغير النتيجة متبوعاً بدالة التنفيذ القياسية بأحرف صغيرة وتمرير رابط الاتصال ومتغير استعلام التحديث مثل `$result = mysql_query($conn, $query);`

```
// تشغيل استعلام التعديل على السيرفر. 1
$result = mysql_query($conn, $query);
```

2. فحص نجاح عملية التعديل بالسيرفر: الانتقال لسطر جديد وبناء جملة الشرط `if` للتحقق من عبور وثبات التغييرات داخل الجدول مثل `if ($result)`

```
// التحقق من نجاح الحفظ. 2
if ($result) {
    // التوجيه التلقائي للمستخدم وإغلاق المسار. 5 و 4 و 3
    header("Location: index.php");
    exit(); // إضافة هذا السطر
}
```

3. فتح حاضة التوجيه واستدعاء الدالة :فتح قوس مجعد {والانتقال لسطر جديد وكتابة دالة التحويل القياسية بأحرف صغيرة headerمتبوعة بفتح قوسين هلاليين ().
4. تحديد وجهة صفحة العرض النهائية بدقة :وضع علامتي تنصيص مزدوجة وتضمين الكلمة الحاكمة Location:متبوعة مباشرة بالاسم الصريح لصفحة العرض (مثل "Location: index.php").
5. إغلاق المسار البرمجي وإتمام الحزمة :غلق القوس الهلالي لدالة التوجيه وإنهاء السطر بالفاصلة المنقوطة ;ثم النزول لسطر جديد وإغلاق القوس المجعد للشرط {}الضمان سلامة التنفيذ في المعمل.

"اسم حقل المعرف = \$متغير الرقم الكودي WHERE 'اسم العمود = '\$متغير القيمة الجديدة SET اسم الجدول UPDATE " = متغير الاستعلام \$

```
);متغير الاتصال, $متغير الاستعلام)mysqli_query( = متغير النتيجة$
```

```
{ (متغير النتيجة)$ if
```

```
header("Location: اسم صفحة العرض.php");
```

```
}
```

```
$result = mysqli_query($conn, $query);
```

```
if ($result) {
```

```
header("Location: index.php");
```

```
}
```

لضمان سلامة التنفيذ وتجنب الأخطاء الشائعة أثناء التطبيقات العملية في المعمل، يُنصح دائماً بكتابة دالة exit()مباشرة بعد دالة header(). هذه الخطوة توقف قراءة المتصفح لأي أكواد PHP إضافية قد تكون موجودة أسفل هذا السطر، مما يوفر أداءً أسرع ويمنع أي تعارضات برمجية غير متوقعة.